

Python Cheat Sheet

A quick reference for Python 3 – variables, types, strings, collections, and control flow

1. Variables & Data Types

Python uses dynamic typing – no need to declare types explicitly.

```
x = 42          # int
pi = 3.14       # float
name = 'Alice'  # str
active = True   # bool
nothing = None  # NoneType
big = 10_000_000 # readable int literal
c = 3 + 4j      # complex
```

■ Use `type()` to check the type of any variable: `type(x) →`

2. Strings

Strings are immutable sequences of Unicode characters.

```
s = 'hello'
s2 = "world"
multi = '''line one
line two'''

# f-strings (Python 3.6+) – preferred formatting
name, age = 'Alice', 30
msg = f'{name} is {age} years old' # 'Alice is 30 years old'
pi_str = f'{3.14159:.2f}'          # '3.14'

# Common string methods
s.upper()      # 'HELLO'
s.strip()      # removes whitespace
s.replace('l', 'r') # 'herro'
s.split(',')   # list of substrings
','.join(['a', 'b', 'c']) # 'a,b,c'
s[1:4]         # slicing: 'ell'
len(s)         # 5
```

3. Collections

3.1 List – ordered, mutable

```
nums = [1, 2, 3, 4, 5]
nums.append(6)    # [1,2,3,4,5,6]
nums.insert(0, 0) # insert at index
nums.pop()        # removes & returns last
nums.sort()       # in-place sort
nums[1:3]         # [2, 3] – slicing
nums[-1]          # last element
# List comprehension
squares = [x**2 for x in range(10)]
evens = [x for x in nums if x % 2 == 0]
```

3.2 Tuple – ordered, immutable

```
point = (10, 20)
x, y = point      # unpacking
single = (42,)    # one-element tuple needs trailing comma
```

```
nested = ((1,2),(3,4))
```

3.3 Dictionary – key-value pairs, ordered (3.7+)

```
person = {'name': 'Alice', 'age': 30}
person['city'] = 'Sofia'      # add key
person.get('age', 0)         # safe access (default 0)
person.keys()                # dict_keys(['name', 'age', 'city'])
person.values()
person.items()               # (key,value) pairs
person.pop('city')           # remove key
# Dict comprehension
squares = {x: x**2 for x in range(5)}
```

3.4 Set – unique, unordered

```
s = {1, 2, 3, 3}             # {1, 2, 3}
s.add(4)
s.discard(1)                 # remove if present
a = {1,2,3}; b = {2,3,4}
a | b                        # union      {1,2,3,4}
a & b                        # intersect {2,3}
a - b                        # difference {1}
```

4. Control Flow

```
# if / elif / else
x = 10
if x > 0:
    print('positive')
elif x == 0:
    print('zero')
else:
    print('negative')

# Ternary expression
label = 'even' if x % 2 == 0 else 'odd'

# for loop
for i in range(5):           # 0 1 2 3 4
    print(i)

for i, val in enumerate(['a','b','c']):
    print(i, val)            # 0 a  1 b  2 c

for k, v in person.items():
    print(k, '->', v)

# while loop
n = 5
while n > 0:
    n -= 1

# break / continue / pass
for x in range(10):
    if x == 5: break          # exit loop
    if x % 2 == 0: continue  # skip iteration
    pass                     # no-op placeholder
```

5. Functions

```
# Basic function
def greet(name, greeting='Hello'):
```

```

    return f'{greeting}, {name}!'

greet('Alice')          # 'Hello, Alice!'
greet('Bob', 'Hi')      # 'Hi, Bob!'

# *args and **kwargs
def total(*args):        # variable positional args
    return sum(args)

def show(**kwargs):      # variable keyword args
    for k, v in kwargs.items():
        print(f'{k}={v}')

# Lambda (anonymous function)
double = lambda x: x * 2
add     = lambda x, y: x + y

# Type hints (Python 3.5+)
def multiply(a: int, b: int) -> int:
    return a * b

```

■ Use type hints for readability and IDE support – they are not enforced at runtime.

6. Object-Oriented Programming (OOP)

```
class Animal:
    species = 'Unknown'          # class variable

    def __init__(self, name, sound):
        self.name = name         # instance variable
        self.sound = sound

    def speak(self):             # instance method
        return f'{self.name} says {self.sound}'

    def __repr__(self):          # string representation
        return f'Animal({self.name!r})'

    @classmethod
    def create_dog(cls):         # factory class method
        return cls('Dog', 'Woof')

    @staticmethod
    def is_animal(obj):          # no self/cls
        return isinstance(obj, Animal)

# Inheritance
class Dog(Animal):
    def __init__(self, name):
        super().__init__(name, 'Woof')

    def fetch(self, item):
        return f'{self.name} fetches {item}'

dog = Dog('Rex')
dog.speak()    # 'Rex says Woof'
dog.fetch('ball')
```

7. Error Handling

```
# try / except / else / finally
try:
    result = 10 / 0
except ZeroDivisionError as e:
    print(f'Error: {e}')
except (TypeError, ValueError) as e:
    print(f'Type or Value error: {e}')
except Exception as e:
    print(f'Unexpected: {e}')
else:
    print('No error occurred') # runs if no exception
finally:
    print('Always runs')      # cleanup code

# Raise custom exceptions
class ValidationError(Exception):
    pass

def validate_age(age):
    if age < 0:
        raise ValidationError(f'Invalid age: {age}')
```

8. File I/O

```
# Reading a file
with open('data.txt', 'r', encoding='utf-8') as f:
    content = f.read()      # entire file as string
    lines   = f.readlines() # list of lines

# Writing a file
with open('output.txt', 'w', encoding='utf-8') as f:
    f.write('Hello, file!') # overwrites

# Append mode
with open('log.txt', 'a') as f:
    f.write('new line\n')

# JSON
import json
data = {'key': 'value', 'num': 42}
json_str = json.dumps(data, indent=2) # dict → JSON string
parsed   = json.loads(json_str)       # JSON string → dict
with open('data.json', 'w') as f:
    json.dump(data, f, indent=2)
```

9. Comprehensions & Generators

```
# List comprehension
squares = [x**2 for x in range(10) if x % 2 == 0]

# Dict comprehension
word_len = {w: len(w) for w in ['python', 'java', 'go']}

# Set comprehension
unique_lens = {len(w) for w in ['hi', 'hello', 'hey']}

# Generator expression (lazy, memory-efficient)
gen = (x**2 for x in range(1_000_000))
next(gen)  # 0 - values computed on demand

# Generator function
def fibonacci():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b

fib = fibonacci()
print([next(fib) for _ in range(8)])  # [0,1,1,2,3,5,8,13]
```

■ Generators are ideal for large datasets – they don't load everything into memory.

10. Useful Built-in Functions

Function	Description	Example
<code>len(x)</code>	Length of sequence	<code>len([1,2,3])</code> → 3
<code>range(n)</code>	Sequence of integers	<code>range(0,10,2)</code> → 0,2,4...
<code>enumerate()</code>	Index + value pairs	<code>enumerate(['a','b'])</code>
<code>zip(a,b)</code>	Pair up iterables	<code>zip([1,2],[3,4])</code> → (1,3),(2,4)
<code>map(f,it)</code>	Apply function to iterable	<code>map(str,[1,2,3])</code>
<code>filter(f,it)</code>	Filter by predicate	<code>filter(None,[0,1,2])</code>
<code>sorted(x)</code>	Return sorted list	<code>sorted([3,1,2])</code> → [1,2,3]
<code>any(it)</code>	True if any element truthy	<code>any([0,0,1])</code> → True
<code>all(it)</code>	True if all elements truthy	<code>all([1,1,1])</code> → True
<code>isinstance()</code>	Type check	<code>isinstance(x, int)</code>

11. Virtual Environments & pip

```
# Create a virtual environment
python -m venv .venv

# Activate (Linux/macOS)
source .venv/bin/activate

# Activate (Windows)
.venv\Scripts\activate

# Install packages
pip install requests numpy pandas
```

```
# Save & restore dependencies
pip freeze > requirements.txt
pip install -r requirements.txt

# Deactivate
deactivate
```

12. Quick Tips

```
# Swap two variables
a, b = b, a

# Unpacking
first, *rest = [1, 2, 3, 4]  # first=1, rest=[2,3,4]

# Walrus operator (3.8+)
if n := len(data) > 10:
    print(f'Too long: {n}')

# Context manager for cleanup
with open('file.txt') as f:
    data = f.read()  # file auto-closed after block

# Chained comparisons
0 < x < 100          # cleaner than x>0 and x<100

# Merge dicts (3.9+)
merged = dict1 | dict2

# Repeat a list
zeros = [0] * 10     # [0,0,0,0,0,0,0,0,0,0]
```